LG Display

AX 전형 사전과제

회로/방호영

LG Display

Contents

- 문제 정의
- AI 활용 전략
- 결과 및 시연

문제 정의 (1/2)

➢테스트 칩 검증

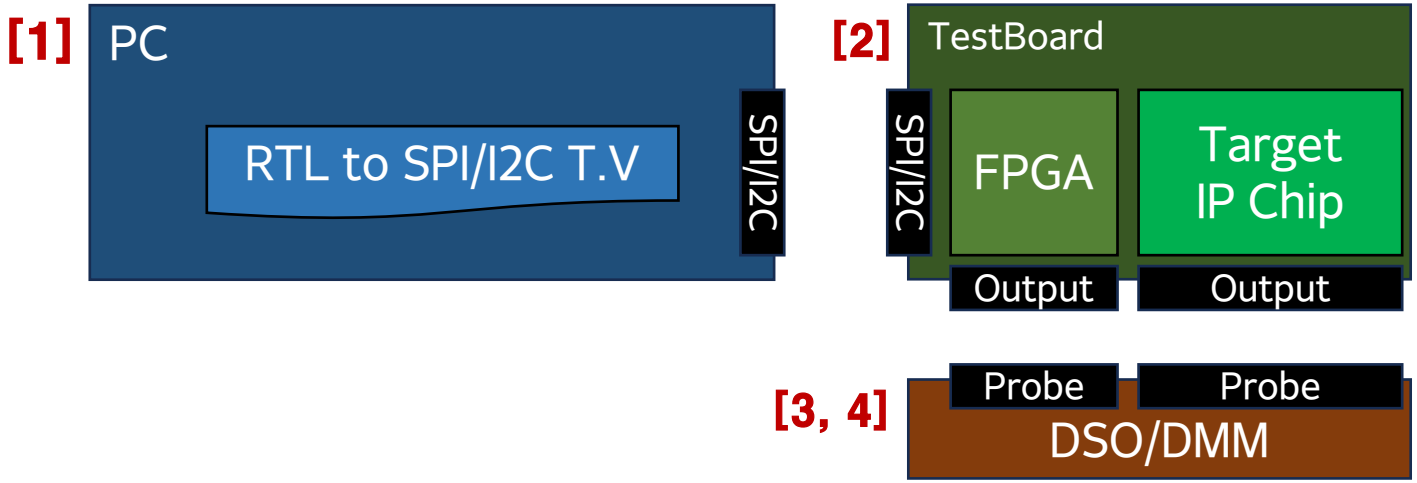
- 기존 테스트 방식 (manual)

  [1] RTL로 작성된 Test Vector를 테스트 보드에 전달하기 위해 SPI/I2C 포맷으로 수정, 신호 전달

  [2] 테스트 보드의 FPGA가 타겟 칩에 신호를 인가

  [3] 오실로스코프/멀티미터를 통해 Digital/Analog 신호를 측정

  [4] 결과 확인 후 해당 Scenario의 Pass/Fail 판단

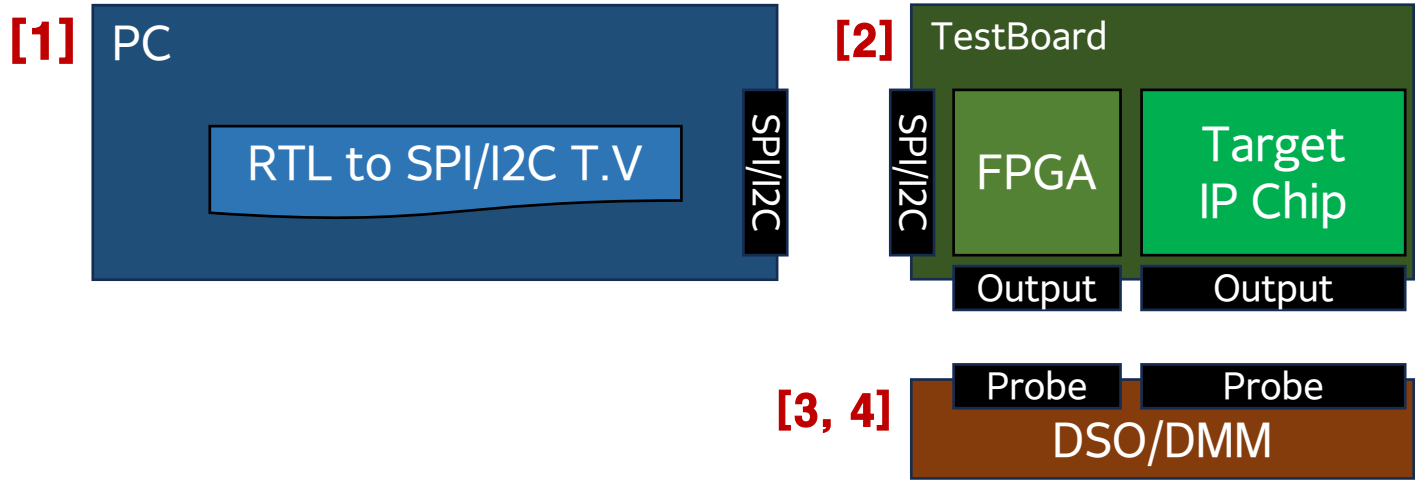


문제 정의 (2/2)

➢테스트 칩 검증

- 기존 방식에서 발생한 문제
  - [1]~[4] 까지의 과정을 수십개의 벡터에 대해 수동으로 진행
  - IP의 종류마다 서로 다른 바이트 순서 (endian)
  - RTL로 작성된 소스코드와 SPI/I2C로 변환된 테스트 벡터의 대응관계를 보기 어려움

→ 위 문제들을 고려하며 [1]~[4]의 과정을 자동화



AI 활용 전략 (1/2)

➢AI를 활용해 해결하려는 이유

- 반복 구조를 가진 다량의 소스코드 분석
  - 여러 RTL 소스코드에 같은 패턴이 반복되어, AI로 패턴을 빠르게 찾을 수 있다고 판단
- 반복되는 테스트 절차의 SW 자동화
  - 변환, 인가, 측정, 판정이 매번 같은 순서로 반복되며, 해당 과정은 SW로 자동화가 가능

*

```
// @test T02 리셋값 확인
// VDDI-100 테스트벡터 (RTL)
`timescale 1ns/1ps
module tv_t02_reset_check;
`include "vddi100_regmap.vh"
`include "tv_tasks.vh"
initial begin
    chip_reset;
    reg_read (ADDR_CHIP_
    reg_read (ADDR_REV_I
    reg_read (ADDR_STATU
    reg_read (ADDR_H_ACT
    reg_read (ADDR_V_ACT
    reg_read (ADDR_DISP_
    reg_read (ADDR_TPG_M
    reg_read (ADDR_FG_R,
    reg_read (ADDR_FG_G,
    reg_read (ADDR_FG_B,
    reg_read (ADDR_CHECK
    reg_read (ADDR_PIX_C
    reg_read (ADDR_SCRAT
    $finish;
```

```
// @test T03 RW 패턴 쓰기/읽기
// VDDI-100 테스트벡터 (RTL)
`timescale 1ns/1ps
module tv_t03_rw_pattern;
`include "vddi100_regmap.vh"
`include "tv_tasks.vh"
initial begin
    chip_reset;
    reg_write(ADDR_H_ACTIVE, 16'h0000);
    reg_read (ADDR_H_ACTIVE, 16'h0000, 16'hFFFF);   // pat=0x0000
    reg_write(ADDR_V_ACTIVE, 16'h0000);
    reg_read (ADDR_V_ACTIVE, 16'h0000, 16'hFFFF);   // pat=0x0000
    reg_write(ADDR_DISP_CTRL, 8'h00);
    reg_read (ADDR_DISP_CTRL, 8'h00, 8'hFF);   // pat=0x00
    reg_write(ADDR_TPG_MODE, 8'h00);
    reg_read (ADDR_TPG_MODE, 8'h00, 8'hFF);   // pat=0x00
    reg_write(ADDR_FG_R, 8'h00);
    reg_read (ADDR_FG_R, 8'h00, 8'hFF);   // pat=0x00
    reg_write(ADDR_FG_G, 8'h00);
```


* 직접 작성한 예제코드 (당시 구현과 무관)

AI 활용 전략 (2/2)

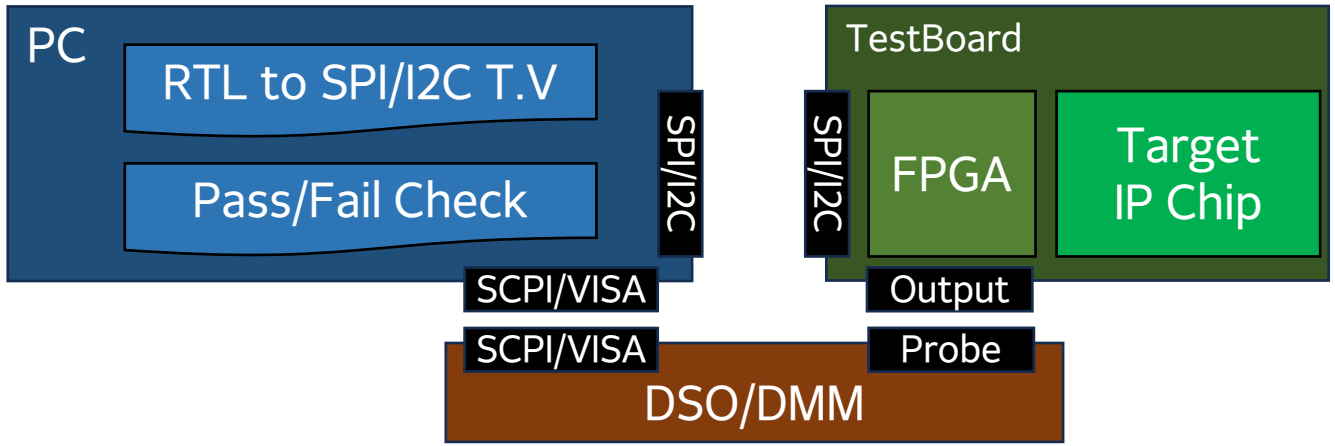
➢AI 활용 방식

[1] 큰 그림은 직접 구성

- 인터페이스 간 연결 관계와 테스트 수행 순서를 직접 구성하고, 간단한 예제로 구현 가능성을 먼저 확인

[2] AI를 이용한 RTL 코드의 패턴 파악 및 소스코드 생성

- 칩 인터페이스(SPI / I2C), 계측기 연결(VISA / SCPI), 팀 Convention, 실행 PC 환경을 전달
- IP별 endian 차이, 프로그램이 사용될 PC별 Python 버전 차이를 요구사항으로 추가
- 단계별 구현/검증이 가능하도록 소스코드 생성



AI 활용 전략 (2/2)

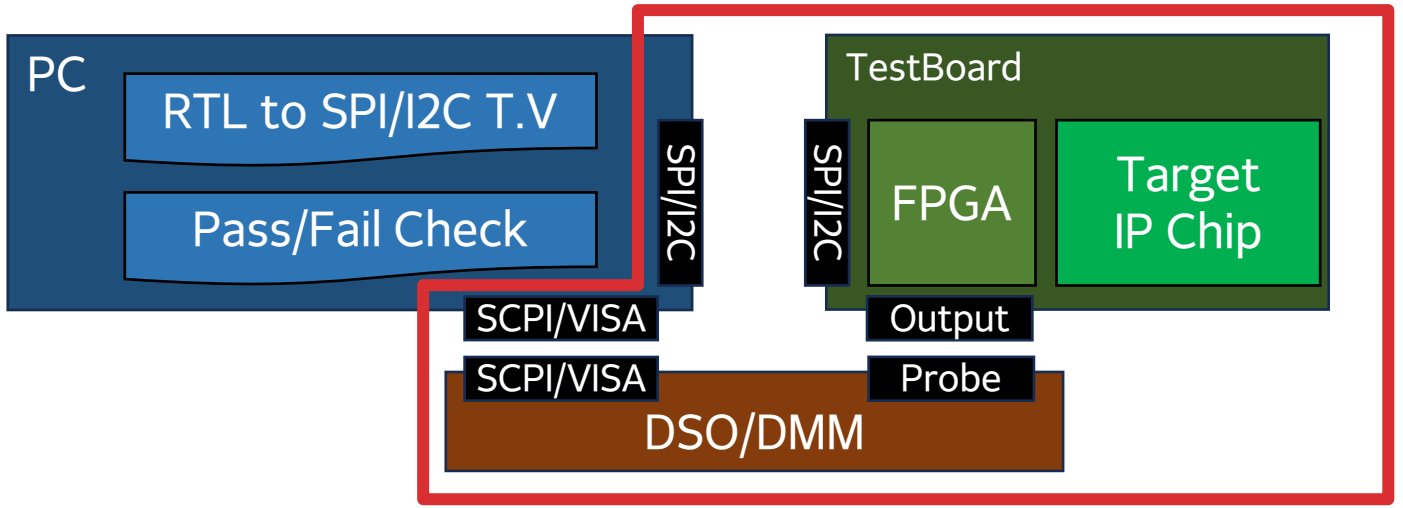
➢AI 활용 방식

[1] 큰 그림은 직접 구성

- 인터페이스 간 연결 관계와 테스트 수행 순서를 직접 구성하고, 간단한 예제로 구현 가능성을 먼저 확인

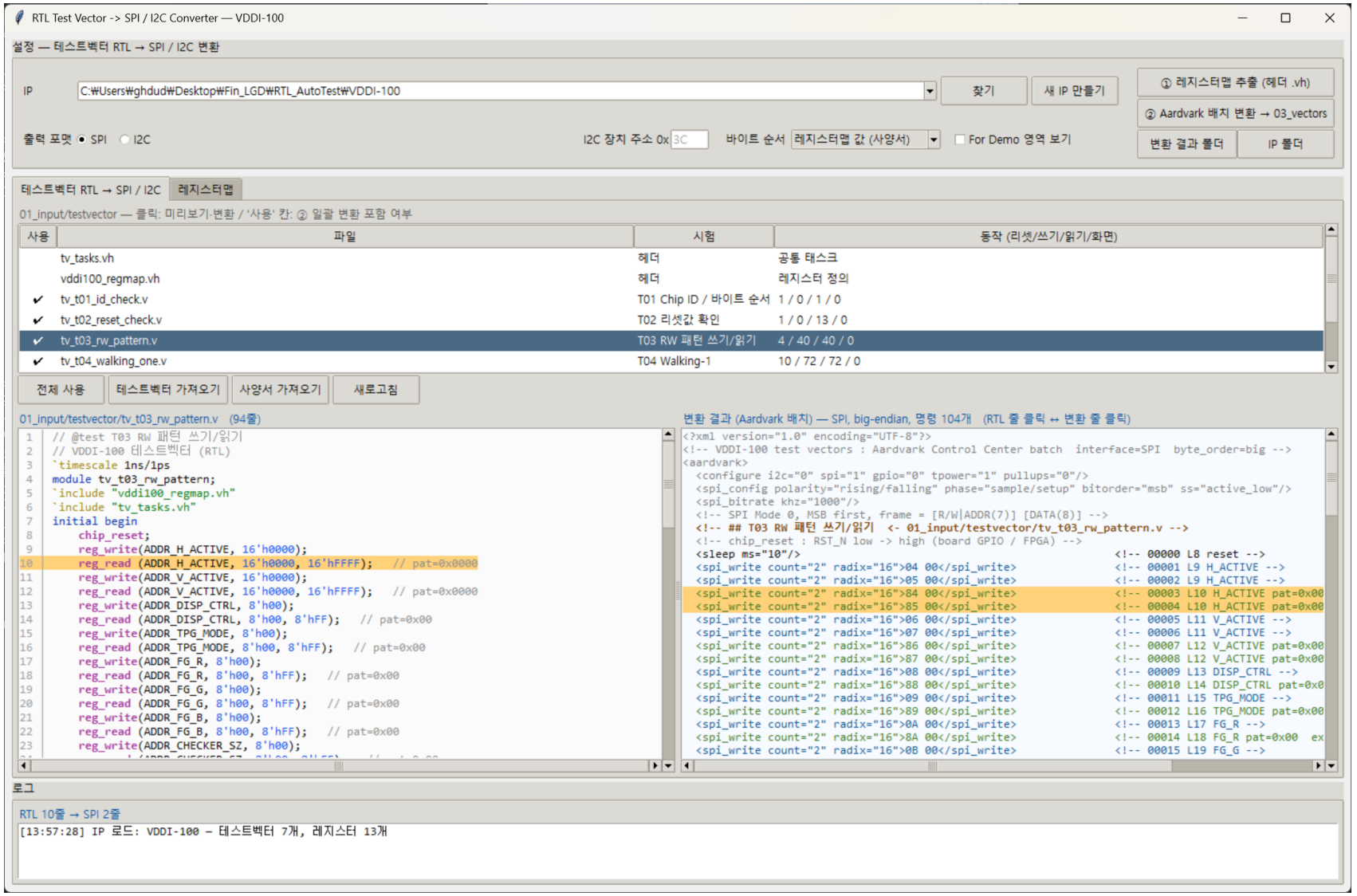
[2] AI를 이용한 RTL 코드의 패턴 파악 및 소스코드 생성

- 칩 인터페이스(SPI / I2C), 계측기 연결(VISA / SCPI), 팀 Convention, 실행 PC 환경을 전달
- IP별 endian 차이, 프로그램이 사용될 PC별 Python 버전 차이를 요구사항으로 추가
- 단계별 구현/검증이 가능하도록 소스코드 생성



* 이번 시연에서는 iverilog RTL 시뮬레이터로 대체

결과 및 시연


RTL Test Vector -> SPI / I2C Converter — VDDI-100
설정 — 테스트벡터 RTL → SPI / I2C 변환
IP
C:₩Users₩ghdud₩Desktop₩Fin_LGD₩RTL_AutoTest₩VDDI-100
찾기
새 IP 만들기
① 레지스터맵 추출 (헤더 .vh)
② Aardvark 배치 변환 → 03_vectors
출력 포맷 SPI I2C
I2C 장치 주소 0x 3C
바이트 순서 레지스터맵 값 (사양서)
For Demo 영역 보기
변환 결과 폴더
IP 폴더
테스트벡터 RTL → SPI / I2C
레지스터맵
01_input/testvector — 클릭: 미리보기·변환 / '사용' 칸: ② 일괄 변환 포함 여부
사용
파일
시험
동작 (리셋/쓰기/읽기/화면)
tv_tasks.vh 헤더 공통 태스크
vddi100_regmap.vh 헤더 레지스터 정의
tv_t01_id_check.v T01 Chip ID / 바이트 순서 1 / 0 / 1 / 0
tv_t02_reset_check.v T02 리셋값 확인 1 / 0 / 13 / 0
tv_t03_rw_pattern.v T03 RW 패턴 쓰기/읽기 4 / 40 / 40 / 0
tv_t04_walking_one.v T04 Walking-1 10 / 72 / 72 / 0
전체 사용
테스트벡터 가져오기
사양서 가져오기
새로고침
01_input/testvector/tv_t03_rw_pattern.v (94줄)
// @test T03 RW 패턴 쓰기/읽기
// VDDI-100 테스트벡터 (RTL)
`timescale 1ns/1ps
module tv_t03_rw_pattern;
`include "vddi100_regmap.vh"
`include "tv_tasks.vh"
initial begin
chip_reset;
reg_write(ADDR_H_ACTIVE, 16'h0000);
reg_read (ADDR_H_ACTIVE, 16'h0000, 16'hFFFF); // pat=0x0000
reg_write(ADDR_V_ACTIVE, 16'h0000);
reg_read (ADDR_V_ACTIVE, 16'h0000, 16'hFFFF); // pat=0x0000
reg_write(ADDR_DISP_CTRL, 8'h00);
reg_read (ADDR_DISP_CTRL, 8'h00, 8'hFF); // pat=0x00
reg_write(ADDR_TPG_MODE, 8'h00);
reg_read (ADDR_TPG_MODE, 8'h00, 8'hFF); // pat=0x00
reg_write(ADDR_FG_R, 8'h00);
reg_read (ADDR_FG_R, 8'h00, 8'hFF); // pat=0x00
reg_write(ADDR_FG_G, 8'h00);
reg_read (ADDR_FG_G, 8'h00, 8'hFF); // pat=0x00
reg_write(ADDR_FG_B, 8'h00);
reg_read (ADDR_FG_B, 8'h00, 8'hFF); // pat=0x00
reg_write(ADDR_CHECKER_SZ, 8'h00);
변환 결과 (Aardvark 배치) — SPI, big-endian, 명령 104개 (RTL 줄 클릭 ↔ 변환 줄 클릭)
<?xml version="1.0" encoding="UTF-8"?>
<!-- VDDI-100 test vectors : Aardvark Control Center batch interface=SPI byte_order=big -->
<aardvark>
<configure i2c="0" spi="1" gpio="0" tpower="1" pullups="0"/>
<spi_config polarity="rising/falling" phase="sample/setup" bitorder="msb" ss="active_low"/>
<spi_bitrate khz="1000"/>
<!-- SPI Mode 0, MSB first, frame = [R/W|ADDR(7)] [DATA(8)] -->
<!-- ## T03 RW 패턴 쓰기/읽기 <- 01_input/testvector/tv_t03_rw_pattern.v -->
<!-- chip_reset : RST_N low -> high (board GPIO / FPGA) -->
<sleep ms="10"/> <!-- 00000 L8 reset -->
<spi_write count="2" radix="16">04 00</spi_write> <!-- 00001 L9 H_ACTIVE -->
<spi_write count="2" radix="16">05 00</spi_write> <!-- 00002 L9 H_ACTIVE -->
<spi_write count="2" radix="16">84 00</spi_write> <!-- 00003 L10 H_ACTIVE pat=0x00
<spi_write count="2" radix="16">85 00</spi_write> <!-- 00004 L10 H_ACTIVE pat=0x00
<spi_write count="2" radix="16">06 00</spi_write> <!-- 00005 L11 V_ACTIVE -->
<spi_write count="2" radix="16">07 00</spi_write> <!-- 00006 L11 V_ACTIVE -->
<spi_write count="2" radix="16">86 00</spi_write> <!-- 00007 L12 V_ACTIVE pat=0x00
<spi_write count="2" radix="16">87 00</spi_write> <!-- 00008 L12 V_ACTIVE pat=0x00
<spi_write count="2" radix="16">08 00</spi_write> <!-- 00009 L13 DISP_CTRL -->
<spi_write count="2" radix="16">88 00</spi_write> <!-- 00010 L14 DISP_CTRL pat=0x0
<spi_write count="2" radix="16">09 00</spi_write> <!-- 00011 L15 TPG_MODE -->
<spi_write count="2" radix="16">89 00</spi_write> <!-- 00012 L16 TPG_MODE pat=0x00
<spi_write count="2" radix="16">0A 00</spi_write> <!-- 00013 L17 FG_R -->
<spi_write count="2" radix="16">8A 00</spi_write> <!-- 00014 L18 FG_R pat=0x00 ex
<spi_write count="2" radix="16">0B 00</spi_write> <!-- 00015 L19 FG_G -->
로그
RTL 10줄 → SPI 2줄
[13:57:28] IP 로드: VDDI-100 — 테스트벡터 7개, 레지스터 13개

LG Display

감사합니다.